الكلية متعددة التخصصات - ورزازات
FACULTÉ POLYDISCIPLINAIRE DE OUARZAZATE

- **Nom et prénom :** BAHHAR Mohamed

- **Groupe** : 1ère année Master IAA

- **Enseignant** : Kamal OMARI

Table des matières

Introduction.....2
Captures (Explication) 3
Conclusion générale.....9

Introduction

Ce TP porte sur le prétraitement des données du dataset Titanic.

L'objectif est de préparer des données propres et exploitables pour un futur modèle de Machine Learning.

Le travail réalise suit 4 étapes principales :

1. Comprendre la structure du dataset (types, aperçu, valeurs manquantes).
2. Nettoyer les données (imputation et traitement des valeurs extrêmes).
3. Transformer les variables (encodage + normalisation/standardisation).
4. Traiter le déséquilibre des classes de la variable cible Survived.

Le notebook utilise Pandas, NumPy, Seaborn, Matplotlib, scikit-learn et SMOTE (avec repli automatique si incompatibilité de version).

Captures (Explication)

```python
# Bibliothèques principales
import warnings
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from IPython.display import display

# Utilitaires scikit-learn
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.utils import resample

# Imbalanced-learn (avec repli automatique si incompatibilité de versions)
try:
    from imblearn.over_sampling import SMOTE as _SMOTE

    class SMOTE(_SMOTE):
        pass
except Exception as exc:
    warnings.warn(
        "SMOTE (imblearn) indisponible ou incompatible. "
```

=> Imports et configuration : Cette cellule charge les bibliothèques nécessaires.

```python
raw_df = sns.load_dataset("titanic")
rename_map = {
    "survived": "Survived",
    "pclass": "Pclass",
    "sex": "Sex",
    "age": "Age",
    "sibsp": "SibSp",
    "parch": "Parch",
    "fare": "Fare",
    "embarked": "Embarked",
    "class": "Class",
    "who": "Who",
    "adult_male": "AdultMale",
    "deck": "Deck",
    "embark_town": "EmbarkTown",
    "alive": "Alive",
    "alone": "Alone"
}
df = raw_df.rename(columns=rename_map).copy()
print(f"Dimensions du jeu de données : {df.shape}")
print("\nTypes de données :")
```

=>Dans cette première étape, le jeu de données Titanic a été chargé à l'aide de la fonction load_dataset() de la bibliothèque Seaborn. Ce dataset est couramment utilisé en Machine Learning pour des tâches de classification, notamment pour prédire la survie des passagers.

Cette étape d'exploration est essentielle, car elle permet de :

- Comprendre la structure du dataset.
- Identifier les types de variables.
- Détecter les valeurs manquantes.

Ces informations sont indispensables pour appliquer correctement les techniques de prétraitement dans les étapes suivantes.

```
Embarked        object
Class         category
Who             object
AdultMale         bool
Deck          category
EmbarkTown      object
Alive           object
Alone             bool
dtype: object
```

Premières lignes :

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	Class	Who	AdultMale	Deck	EmbarkTown	Alive	Alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

=>Cette cellule donne une vue globale du dataset Titanic et identifie les premiers problèmes de qualité (valeurs manquantes).

```python
df_clean = df.copy()
missing_ratio = df_clean.isnull().mean().sort_values(ascending=False)
print("Proportion de valeurs manquantes par colonne :")
display(missing_ratio.to_frame(name="ratio"))
cols_to_drop = missing_ratio[missing_ratio > 0.40].index.tolist()
print(f"\nColonnes supprimées (>40 % manquantes) : {cols_to_drop}")
df_clean = df_clean.drop(columns=cols_to_drop)
print(f"Forme restante : {df_clean.shape}")
```

Python

Proportion de valeurs manquantes par colonne :

	ratio
Deck	0.772166
Age	0.198653
Embarked	0.002245
EmbarkTown	0.002245
Survived	0.000000
Pclass	0.000000
Sex	0.000000
SibSp	0.000000
Parch	0.000000
Fare	0.000000
Class	0.000000
Who	0.000000
AdultMale	0.000000
Alive	0.000000
Alone	0.000000

```
Colonnes supprimées (>40 % manquantes) : ['Deck']
Forme restante : (891, 14)
```

Generate + Code + Markdown

=>Cette cellule explique la stratégie de nettoyage : suppression des colonnes trop manquantes, imputation et traitement des valeurs extrêmes.

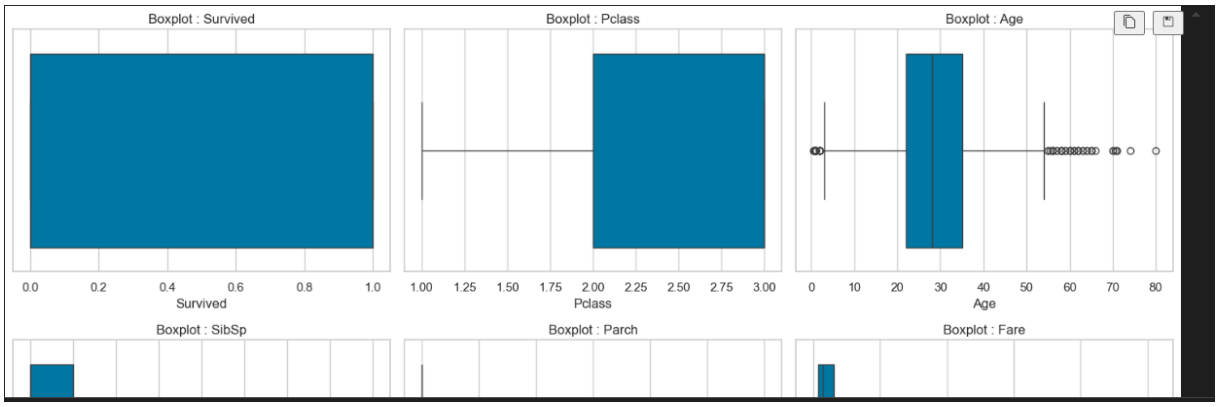
Cette étape de nettoyage est essentielle pour améliorer la qualité du dataset. En supprimant les colonnes fortement incomplètes, on garantit que les données utilisées pour l'analyse et la modélisation sont plus cohérentes et exploitables.

```python
# Imputation : médiane pour le numérique, mode pour le catégoriel
numeric_cols = df_clean.select_dtypes(include=[np.number]).columns.tolist()
categorical_cols = df_clean.select_dtypes(exclude=[np.number]).columns.tolist()
for col in numeric_cols:
    median_val = df_clean[col].median()
    df_clean[col].fillna(median_val, inplace=True)
for col in categorical_cols:
    mode_val = df_clean[col].mode(dropna=True)
    if not mode_val.empty:
```

```
Valeurs manquantes restantes par colonne :
Survived      0
Pclass        0
Sex           0
Age           0
SibSp         0
Parch         0
Fare          0
Embarked      0
Class         0
Who           0
```

=>Après avoir supprimé les colonnes contenant un grand nombre de valeurs manquantes, une étape d'imputation a été réalisée afin de traiter les valeurs manquantes restantes dans le dataset. Les variables numériques sont imputées par la médiane, les variables catégorielles par le mode

```python
# Visualisation des valeurs extrêmes via boxplots
num_plots = len(numeric_cols)
cols = 3
rows = int(np.ceil(num_plots / cols))
plt.figure(figsize=(cols * 5, rows * 4))
for idx, col in enumerate(numeric_cols, start=1):
    plt.subplot(rows, cols, idx)
    sns.boxplot(x=df_clean[col], color="#1f77b4")
    plt.title(f"Boxplot : {col}")
plt.tight_layout()
plt.show()
```



=>L'utilisation des boxplots constitue une étape essentielle dans l'analyse exploratoire des données. Elle permet de détecter efficacement les valeurs aberrantes et de préparer leur traitement dans l'étape suivante, afin d'améliorer la qualité du dataset.

```python
# Remplacement des valeurs extrêmes (au-delà de 1,5 IQR) par la médiane
for col in numeric_cols:
    q1 = df_clean[col].quantile(0.25)
    q3 = df_clean[col].quantile(0.75)
    iqr = q3 - q1
    if iqr == 0:
        continue
    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr
    median_val = df_clean[col].median()
    mask = (df_clean[col] < lower_bound) | (df_clean[col] > upper_bound)
    replaced = mask.sum()
    df_clean.loc[mask, col] = median_val
    print(f"{col} : {replaced} valeurs extrêmes remplacées par la médiane")
```

Python

```
Survived : 0 valeurs extrêmes remplacées par la médiane
Pclass : 0 valeurs extrêmes remplacées par la médiane
Age : 66 valeurs extrêmes remplacées par la médiane
SibSp : 46 valeurs extrêmes remplacées par la médiane
Fare : 116 valeurs extrêmes remplacées par la médiane
```

=>Après avoir identifié les valeurs aberrantes à l'aide des boxplots, une méthode statistique basée sur l'**intervalle interquartile (IQR)** a été utilisée pour les traiter. L'utilisation de la méthode IQR combinée au remplacement par la médiane permet de traiter efficacement les valeurs aberrantes sans supprimer de données. Cette approche améliore la qualité du dataset et prépare les données pour les étapes de transformation et de modélisation.

```python
# Encodage One-Hot des variables nominales (Pclass reste ordinale)
target_col = "Survived"
categorical_cols = df_clean.select_dtypes(include=["object", "category", "bool"]).columns.tolist()
nominal_cols = categorical_cols.copy()
df_encoded = pd.get_dummies(df_clean, columns=nominal_cols, drop_first=True)
print(f"Dimensions après encodage : {df_encoded.shape}")
display(df_encoded.head())
```

Python

```
Dimensions après encodage : (891, 18)
```

	Survived	Pclass	Age	SibSp	Parch	Fare	Sex_male	Embarked_Q	Embarked_S	Class_Second	Class_Third	Who_man	Who_woman	AdultMale_True	EmbarkTown_Q
0	0	3	22.0	1	0	7.2500	True	False	True	False	True	True	False	True	
1	1	1	38.0	1	0	14.4542	False	False	False	False	False	False	True	False	
2	1	3	26.0	0	0	7.9250	False	False	True	False	True	False	True	False	
3	1	1	35.0	1	0	53.1000	False	False	True	False	False	False	True	False	
4	0	3	35.0	0	0	8.0500	True	False	True	False	True	True	False	True	

=>L'encodage des variables catégorielles constitue une étape essentielle du prétraitement. Le One-Hot Encoding permet de transformer efficacement les données qualitatives en données numériques, tout en évitant les biais liés à une interprétation incorrecte des catégories.

```python
num_features = [col for col in df_clean.select_dtypes(include=[np.number]).columns if col != target_col]
minmax_scaler = MinMaxScaler()
standard_scaler = StandardScaler()
df_minmax = df_clean.copy()
df_standard = df_clean.copy()
df_minmax[num_features] = minmax_scaler.fit_transform(df_minmax[num_features])
df_standard[num_features] = standard_scaler.fit_transform(df_standard[num_features])
display(df_minmax.head())
display(df_standard.head())
```

Python

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	Class	Who	AdultMale	EmbarkTown	Alive	Alone
0	0	1.0	male	0.372549	0.5	0.0	0.111538	S	Third	man	True	Southampton	no	False
1	1	0.0	female	0.686275	0.5	0.0	0.222372	C	First	woman	False	Cherbourg	yes	False
2	1	1.0	female	0.450980	0.0	0.0	0.121923	S	Third	woman	False	Southampton	yes	True
3	1	0.0	female	0.627451	0.5	0.0	0.816923	S	First	woman	False	Southampton	yes	False
4	0	1.0	male	0.627451	0.0	0.0	0.123846	S	Third	man	True	Southampton	no	True

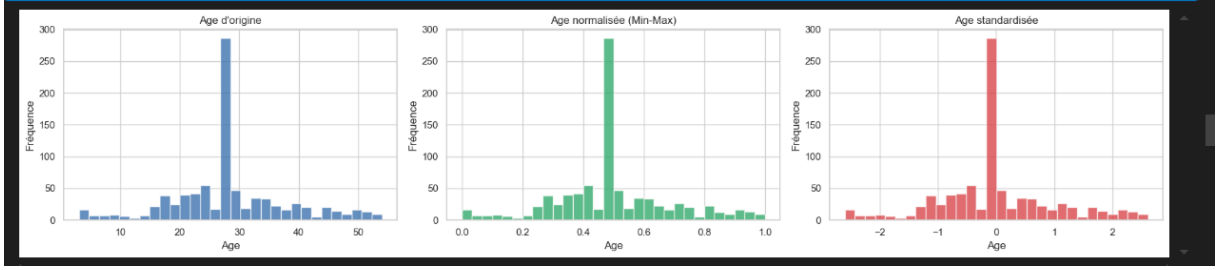
	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	Class	Who	AdultMale	EmbarkTown	Alive	Alone
0	0	0.827377	male	-0.661724	1.347605	-0.473674	-0.797554	S	Third	man	True	Southampton	no	False
1	1	-1.566107	female	0.972921	1.347605	-0.473674	-0.230556	C	First	woman	False	Cherbourg	yes	False
2	1	0.827377	female	-0.253063	-0.570472	-0.473674	-0.744429	S	Third	woman	False	Southampton	yes	True

=>Dans cette étape, les variables numériques du dataset ont été transformées à l'aide de deux techniques de mise à l'échelle : la **normalisation (Min-Max)** et la **standardisation (Z-score)**. La normalisation et la standardisation sont des étapes essentielles du prétraitement. Elles permettent

d'améliorer la qualité des données et d'optimiser les performances des modèles de Machine Learning, en particulier pour les algorithmes basés sur les distances ou les gradients.

```python
feature_to_plot = "Age" if "Age" in df_clean.columns else num_features[0]
fig, axes = plt.subplots(1, 3, figsize=(18, 4))
axes[0].hist(df_clean[feature_to_plot], bins=30, color="#4c72b0", alpha=0.8)
axes[0].set_title(f"{feature_to_plot} d'origine")
axes[1].hist(df_minmax[feature_to_plot], bins=30, color="#55a868", alpha=0.8)
axes[1].set_title(f"{feature_to_plot} normalisée (Min-Max)")
axes[2].hist(df_standard[feature_to_plot], bins=30, color="#c44e52", alpha=0.8)
axes[2].set_title(f"{feature_to_plot} standardisée")
for ax in axes:
    ax.set_xlabel(feature_to_plot)
    ax.set_ylabel("Fréquence")
plt.tight_layout()
plt.show()
```

Python



=>Dans cette étape, une comparaison visuelle des distributions a été réalisée pour une variable numérique représentative (par défaut **Age**), avant et après application des techniques de normalisation et de standardisation.

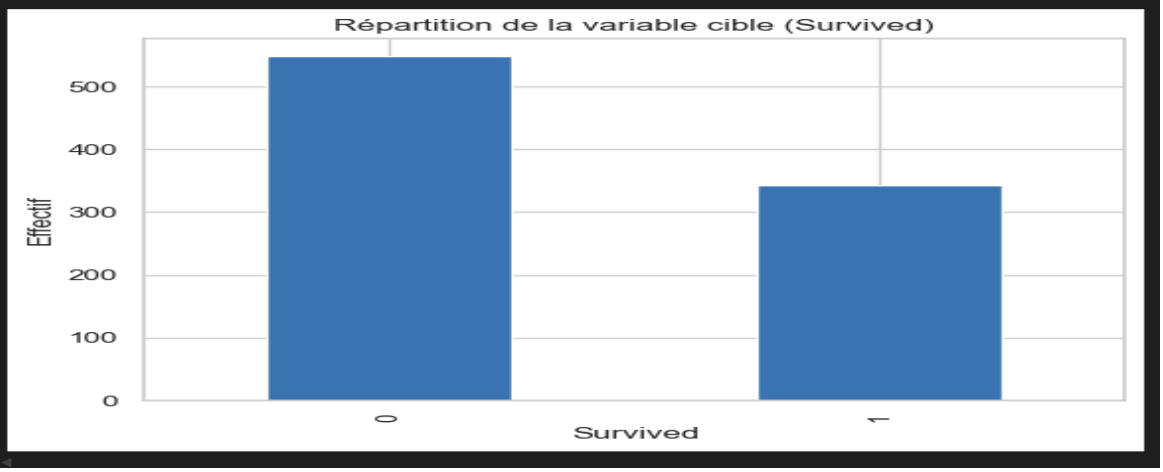
Trois histogrammes ont été tracés :

- La distribution originale.
- La distribution après normalisation (Min-Max).
- La distribution après standardisation (Z-score).

```python
print(df_clean[target_col].value_counts())
df_clean[target_col].value_counts().plot(kind="bar", title="Répartition de la variable cible (Survived)")
plt.xlabel("Survived")
plt.ylabel("Effectif")
plt.show()
```

Python

```
0    549
1    342
Name: count, dtype: int64
```



=>Dans cette étape, la distribution de la variable cible **Survived** a été analysée afin de vérifier si le dataset est équilibré ou non.

La fonction value_counts() a été utilisée pour compter le nombre d'observations dans chaque classe, puis un graphique en barres a été tracé pour visualiser cette répartition.

```python
# Sous-échantillonnage de la classe majoritaire
df_majority = df_encoded[df_encoded[target_col] == 0]
df_minority = df_encoded[df_encoded[target_col] == 1]
df_majority_downsampled = resample(
    df_majority,
    replace=False,
    n_samples=len(df_minority),
    random_state=42
)
df_undersampled = pd.concat([df_majority_downsampled, df_minority])
print("Répartition après sous-échantillonnage :")
print(df_undersampled[target_col].value_counts())
```

Python

```
Répartition après sous-échantillonnage :
Survived
0    342
1    342
Name: count, dtype: int64
```

=>Le sous-échantillonnage a permis d'équilibrer les classes en réduisant la taille de la classe majoritaire. Cette méthode améliore la capacité du modèle à apprendre les deux classes, mais peut entraîner une perte d'information.

```python
# Sur-échantillonnage avec SMOTE
X = df_encoded.drop(columns=[target_col])
y = df_encoded[target_col]
smote = SMOTE(random_state=42)
X_smote, y_smote = smote.fit_resample(X, y)
print("Répartition équilibrée via SMOTE :")
print(pd.Series(y_smote).value_counts())
```

Python

```
Répartition équilibrée via SMOTE :
0    549
1    549
Name: count, dtype: int64
```

=>Dans cette étape, une technique avancée de rééquilibrage des classes appelée **SMOTE (Synthetic Minority Over-sampling Technique)** a été appliquée.

Contrairement au sous-échantillonnage, cette méthode consiste à **augmenter artificiellement le nombre d'exemples de la classe minoritaire**, sans supprimer de données existantes.

Conclusion générale

Le TP montre un pipeline complet de prétraitement pour un problème de classification.

Les principales étapes du prétraitement des données en Machine Learning sont essentielles pour :

- Améliorer la qualité des données.
- Réduire les biais.
- Optimiser les performances des modèles.

Un bon prétraitement permet d'obtenir un modèle plus fiable, plus précis et capable de mieux généraliser sur de nouvelles données.